Quick revision sheets

25B. Quick Revision Sheets

Items: 11 **Purpose:** One-page cheat sheets for the night before an interview.

25.6 JavaScript Cheat Sheet

JAVASCRIPT Copy

```
SCOPE & CLOSURES
  • var = function-scoped, let/const = block-scoped
  • Closures: function + lexical environment. Inner fn accesses outer vars
  • Temporal Dead Zone: let/const exist but can't be accessed before decla

  THIS KEYWORD
  • Default: window (strict: undefined)
  • Object method: the object
  • Arrow fn: inherits from enclosing scope
  • new: the new instance
  • call/apply/bind: explicitly set

  ASYNC
  • Event loop: Call stack → Microtask queue (Promises) → Macrotask queue
  • Microtasks always run before macrotasks
  • Promise.all: fails if ANY rejects  |  Promise.allSettled: never fails
  • Promise.race: first to settle      |  Promise.any: first to fulfill

  GOTCHAS
  • 0.1 + 0.2 =/= 0.3 (floating point)
  • typeof null === "object" (JS bug)
```

25.7 React Cheat Sheet

JAVASCRIPT Copy

```
HOOKS QUICK REFERENCE
  useState     → local state, [value, setter]
  useEffect    → side effects, cleanup, deps array
  useRef       → mutable value, persists across renders, no re-render
  useMemo      → cache computation, recompute on deps change
  useCallback  → cache function reference, useful with React.memo
  useReducer   → complex state with dispatch
  useContext   → read context, all consumers re-render on change
  useTransition→ mark update as non-urgent

  RENDERING RULES
  • Re-render: state change, parent re-render, context change
  • Prevent: React.memo, useMemo/useCallback
  • Keys: stable + unique. Never index for dynamic lists

  PATTERNS
  • Controlled: value + onChange (React owns state)
  • Uncontrolled: ref + defaultValue (DOM owns state)
  • Custom hooks: reusable logic, must start with "use"
```

25.8 CSS Cheat Sheet

JAVASCRIPT Copy

```
FLEXBOX
  justify-content: center | space-between | space-around  (main axis)
  align-items: center | stretch | flex-start               (cross axis)
  flex: 1 (grow equally) | flex: none (don't grow/shrink)
```

```
gap: 16px

GRID
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))  ← responsiv
grid-template-areas: "header header" "sidebar main" "footer footer"

SPECIFICITY (high → low)
!important > inline > #id > .class/:pseudo > element
:where() = 0  |  :is() = highest arg  |  :has() = arg specificity

MODERN CSS
container queries, @layer, :has(), clamp(), dvh/svh/lvh, nesting

PERFORMANCE
transform + opacity = GPU (fast) | width/height/top = layout (slow)
content-visibility: auto → skip offscreen rendering
```

25.9 TypeScript Cheat Sheet

JAVASCRIPT Copy

```
ESSENTIALS
  type vs interface: type for unions, interface for object shapes + extend
  Enums → use string unions: type Status = "loading" | "success" | "error"
  as const → readonly + literal types  |  satisfies → validate without wid

  TYPE NARROWING
  typeof, instanceof, "prop" in obj, discriminated unions

  UTILITY TYPES
  Partial<T>  Required<T>  Pick<T,K>  Omit<T,K>  Record<K,V>
  Readonly<T>  ReturnType<typeof fn>  Parameters<typeof fn>

  REACT + TS
  Props: interface ButtonProps { label: string; onClick: () => void }
  Events: React.ChangeEvent<HTMLInputElement>
  Refs: useRef<HTMLInputElement>(null)
```

25.10 Data Structures — Complexity Summary

JAVASCRIPT Copy

```
Structure     | Access | Search | Insert | Delete | Frontend Use
--------------|--------|--------|--------|--------|-------------
Array         | O(1)   | O(n)   | O(n)*  | O(n)*  | Lists, grids
Map/Object    | O(1)   | O(1)   | O(1)   | O(1)   | Lookups, cache
Set           | -      | O(1)   | O(1)   | O(1)   | Dedup, membership
Stack         | O(n)   | O(n)   | O(1)   | O(1)   | Undo/redo
Queue         | O(n)   | O(n)   | O(1)   | O(1)   | Task scheduling
Linked List   | O(n)   | O(n)   | O(1)** | O(1)** | LRU cache
BST           |O(logn) |O(logn) |O(logn) |O(logn) | Sorted data
Heap          | O(1)   | O(n)   |O(logn) |O(logn) | Top-K, priority
Trie          | O(m)   | O(m)   | O(m)   | O(m)   | Autocomplete

* push/pop O(1), shift/splice O(n)  ** with node reference
```

25.11 Algorithm Patterns

JAVASCRIPT Copy

```
Pattern            | Recognize When ...                          | Example
-------------------|---------------------------------------------|----------
Two Pointer        | Sorted array, pair finding                  | Two Sum sort
Sliding Window     | Contiguous subarray/substring               | Max sum K
Binary Search      | Sorted data, min/max optimization           | Rotated arra
BFS                | Shortest path, level-order                  | Rotten oran
DFS                | All paths, tree traversal                   | Number of i
Dynamic Programming| Overlapping subproblems                     | Coin change
Greedy             | Local optimal = global optimal              | Activity se
Monotonic Stack    | Next greater/smaller element               | Daily tempe
Topological Sort   | Dependency ordering                         | Course sche
Union-Find         | Connected components                        | Number of p
```

JAVASCRIPT Copy

```
Module      → Encapsulate code, expose public API
  Singleton  → One instance (ES Module = natural singleton)
  Factory    → Create objects without specifying exact class
  Observer   → Notify dependents when state changes (DOM events)
  Pub/Sub    → Decoupled communication via message broker
  Strategy   → Swap algorithms at runtime (validation, sorting)
  Command    → Encapsulate actions for undo/redo
  Decorator  → Add behavior without modifying original (HOCs)
  Proxy      → Intercept access (validation, Vue reactivity)
  State      → Change behavior based on state (XState)
  Composite  → Tree structure, uniform interface (React tree)
```

25.13 System Design — RADIO Template

JAVASCRIPT Copy

```
R — REQUIREMENTS (5 min)
      Functional: what the user does
      Non-functional: scale, performance, offline, a11y

  A — ARCHITECTURE (10 min)
      Component tree, page structure, routing
      Draw a diagram

  D — DATA MODEL (10 min)
      State shape, API contracts, local vs server vs URL state

  I — INTERFACE / API (5 min)
      Component props, REST/GraphQL/WS endpoints

  O — OPTIMIZATION (10 min)
      Performance, accessibility, error handling, offline
      End with TRADE-OFFS: "I chose X over Y because ... "
```

JAVASCRIPT Copy

```
Attack       | Prevention
  ─────────────┼──────────────────────────────────────
  XSS          | Output encoding, CSP, HttpOnly cookies, DOMPurify
  CSRF         | SameSite cookies, CSRF tokens, check Origin header
  Clickjacking | frame-ancestors CSP, X-Frame-Options
  Supply Chain | Lockfiles, npm audit, exact versions, SRI

  MUST-SET HEADERS
  Content-Security-Policy, Strict-Transport-Security,
  X-Content-Type-Options, Referrer-Policy, Permissions-Policy
```

25.15 Performance Checklist

JAVASCRIPT Copy

```
LOADING
  [ ] Critical CSS inlined           [ ] Scripts deferred
  [ ] Fonts preloaded + swap         [ ] Hero image fetchpriority="high"
  [ ] Images: WebP/AVIF, srcset, lazy [ ] Code split per route
  [ ] Third-party scripts async

  RUNTIME
  [ ] No layout thrashing            [ ] Animations: transform/opacity onl
  [ ] Large lists virtualized        [ ] React.memo on expensive component
  [ ] content-visibility: auto

  NETWORK
  [ ] HTTP/2+, Brotli                [ ] Cache-Control: immutable for hash
  [ ] preconnect to API/CDN          [ ] AbortController for stale request

  MONITORING
  [ ] Core Web Vitals tracked        [ ] Lighthouse CI in pipeline
  [ ] Bundle size budget enforced    [ ] Error tracking (Sentry)
```

25.16 HTTP Reference Card

JAVASCRIPT Copy

```
METHODS
  GET (read) | POST (create) | PUT (replace) | PATCH (partial) | DELETE (r

  STATUS CODES
  200 OK           | 400 Bad Request      | 500 Internal Server Error
  201 Created      | 401 Unauthorized     | 502 Bad Gateway
  204 No Content   | 403 Forbidden        | 503 Service Unavailable
  301 Moved        | 404 Not Found        |
  304 Not Modified | 429 Too Many Requests|

  KEY HEADERS
  Content-Type: application/json     | Cache-Control: public, max-age=3153
  Authorization: Bearer <token>      | Set-Cookie: session=x; HttpOnly; Se
  Accept-Encoding: gzip, br          | Access-Control-Allow-Origin: https:/
```

QUICK CHECK — no marks, just for you.

1. The JavaScript cheat sheet says microtasks always run before macrotasks. What does this snippet log?

JAVASCRIPT Copy

```
console.log('A');
setTimeout(function () { console.log('B'); }, 0);
Promise.resolve().then(function () { console.log('C'); });
console.log('D');
```

- ○ A D B C
- ○ A D

- A B C D
- A C D B

Check answer

2. According to the CSS cheat sheet, what specificity does the :where() pseudo-class contribute to a selector?

- Zero — :where() adds no specificity at all
- The specificity of its lowest-specificity argument
- The specificity of its highest-specificity argument
- The same as a class selector, regardless of its arguments

Check answer

3. Per the web security cheat sheet, which set of measures is listed as the prevention for CSRF?

- frame-ancestors CSP and X-Frame-Options
- Lockfiles, npm audit, exact versions, and SRI
- SameSite cookies, CSRF tokens, and checking the Origin header
- Output encoding, CSP, HttpOnly cookies, and DOMPurify

Previous
Interview guide

LEARN MORE

ARTICLES & DOCS

- JS interview questions (repo)
  github.com
- The Modern JavaScript Tutorial
  javascript.info

VIDEOS



- JavaScript revision & interview cheat sheet
  YouTube



- The only JavaScript cheat sheet you need
  YouTube



- DSA cheat sheet for coding interviews
  YouTube

Hand-picked references for this lesson.